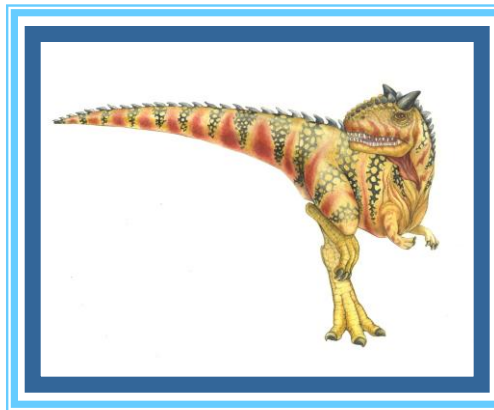


# Chapter 7: Deadlocks

---





# Chapter 7: Deadlocks

---

- System Model
- Deadlock Characterization
- Methods for Handling Deadlocks
- Deadlock Prevention
- Deadlock Avoidance





# System Model

- System consists of resources
- Resource types  $R_1, R_2, \dots, R_m$ 
  - CPU cycles, memory space, I/O devices*
- Each resource type  $R_i$  has  $W_i$  instances.
- Under the normal mode of operation, a process may utilize a resource in only the following sequence
  - **Request** - The process requests the resource. If the request cannot be granted immediately (for example, if the resource is being used by another process), then the requesting process must wait until it can acquire the resource
  - **Use** - The process can operate on the resource (for example, if the resource is a printer, the process can print on the printer)
  - **Release** - The process releases the resource





# Deadlock

- A set of processes is in a deadlocked state when every process in the set is waiting for an event that can be caused only by another process in the set
- Events with which we are mainly concerned here are resource acquisition and release.
- The resources may be either physical resources (for example, printers, tape drives, memory space, and CPU cycles) or logical resources (for example, files, semaphores, and monitors)





# Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- **Mutual exclusion:** only one process at a time can use a resource
- **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes
- **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task
- **Circular wait:** there exists a set  $\{P_0, P_1, \dots, P_n\}$  of waiting processes such that  $P_0$  is waiting for a resource that is held by  $P_1$ ,  $P_1$  is waiting for a resource that is held by  $P_2$ , ...,  $P_{n-1}$  is waiting for a resource that is held by  $P_n$ , and  $P_n$  is waiting for a resource that is held by  $P_0$ .





# Resource-Allocation Graph

Deadlocks can be described more precisely in terms of a directed graph called a system **resource-allocation graph**

A set of vertices  $V$  and a set of edges  $E$ .

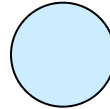
- $V$  is partitioned into two types:
  - $P = \{P_1, P_2, \dots, P_n\}$ , the set consisting of all the processes in the system
  - $R = \{R_1, R_2, \dots, R_m\}$ , the set consisting of all resource types in the system
- **request edge** – directed edge  $P_i \rightarrow R_j$
- **assignment edge** – directed edge  $R_j \rightarrow P_i$





# Resource-Allocation Graph (Cont.)

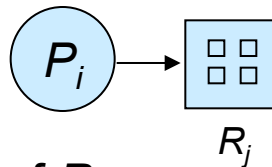
- Process



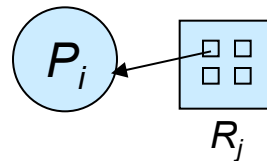
- Resource Type with 4 instances

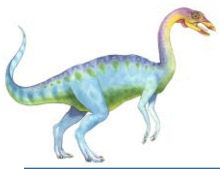


- $P_i$  requests instance of  $R_j$

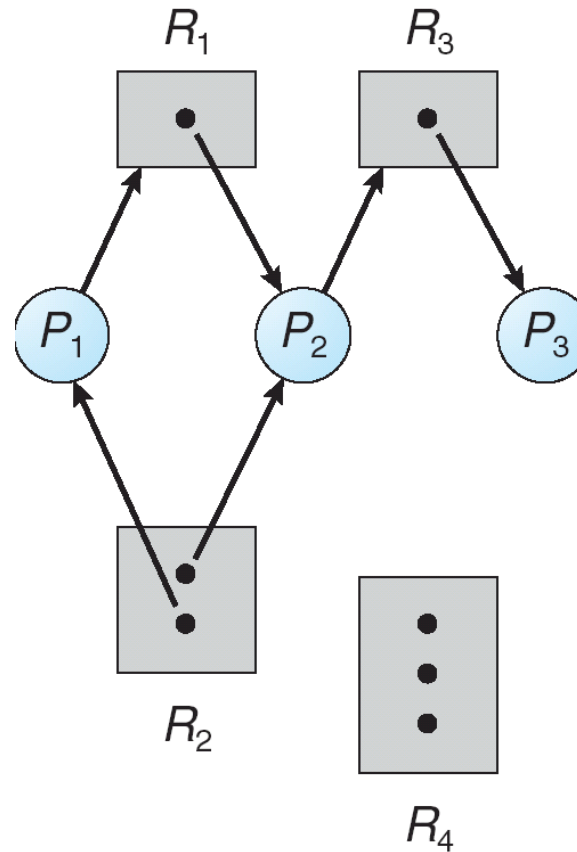


- $P_i$  is holding an instance of  $R_j$





# Example of a Resource Allocation Graph







# Depiction of RAG

## □ The sets P, R, and E:

$$P = \{P1, P2, P3\}$$

$$R = \{R1, R2, R3, R4\}$$

$$E = \{P1 \rightarrow R1, P2 \rightarrow R3, R1 \rightarrow P2, R2 \rightarrow P2, R2 \rightarrow P1, R3 \rightarrow P3\}$$

## □ Resource instances:

One instance of resource type R1

Two instances of resource type R2

One instance of resource type R3

Three instances of resource type R4

## □ Process states:

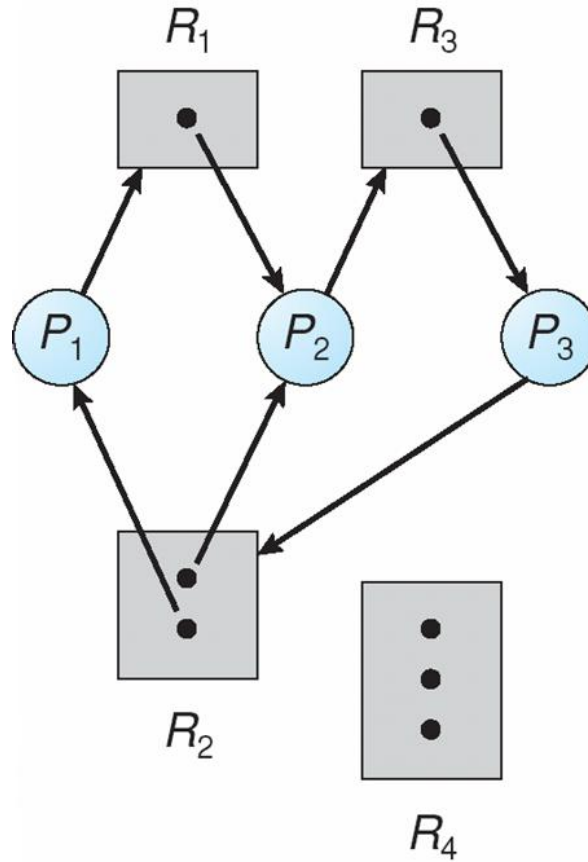
- Process P1 is holding an instance of resource type R2 and is waiting for an instance of resource type R1.
- Process P2 is holding an instance of R1 and an instance of R2 and is waiting for an instance of R3.
- Process P3 is holding an instance of R3.

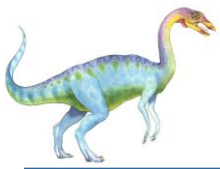
□ if the graph contains no cycles, then no process in the system is deadlocked.



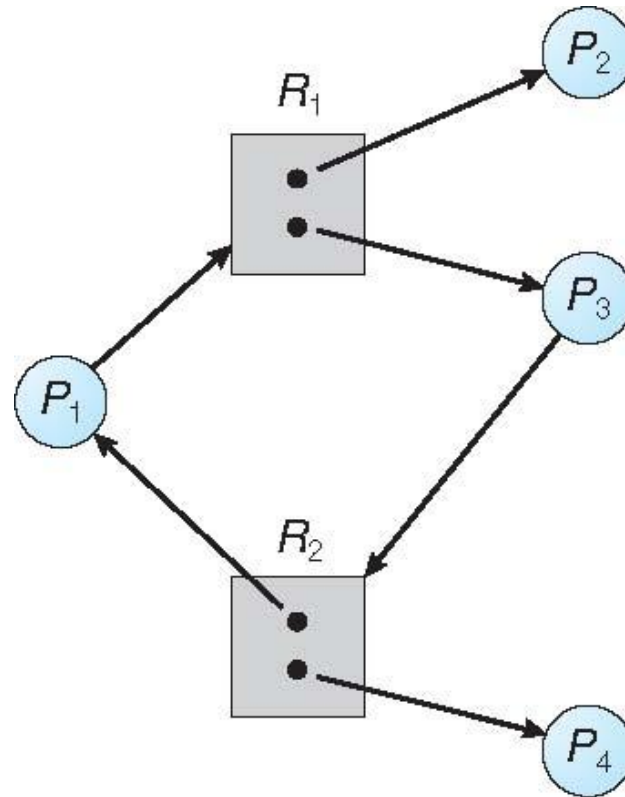


# Resource Allocation Graph With A Deadlock





# Graph With A Cycle But No Deadlock





# Basic Facts

---

- If graph contains no cycles  $\Rightarrow$  no deadlock
- If graph contains a cycle  $\Rightarrow$ 
  - if only one instance per resource type, then deadlock
  - if several instances per resource type, possibility of deadlock





# Methods for Handling Deadlocks

---

- Ensure that the system will **never** enter a deadlock state:
  - Deadlock prevention
  - Deadlock avoidance
- Allow the system to enter a deadlock state and then recover
- Ignore the problem and pretend that deadlocks never occur in the system; used by most operating systems, including UNIX and Windows
  - up to the application developer to write programs that handle deadlocks





# Deadlock Prevention

---

Restrain the ways request can be made

- ❑ **Mutual Exclusion** – not required for sharable resources (e.g., read-only files); must hold for non-sharable resources
- ❑ **Hold and Wait** – must guarantee that whenever a process requests a resource, it does not hold any other resources
  - ❑ Require process to request and be allocated all its resources before it begins execution, or allow process to request resources only when the process has none allocated to it.
  - ❑ Low resource utilization; starvation possible





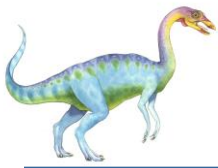
# Deadlock Prevention (Cont.)

## □ No Preemption –

- If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released
- Preempted resources are added to the list of resources for which the process is waiting
- Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting

## □ Circular Wait – impose a total ordering of all resource types, and require that each process requests resources in an increasing order of enumeration





# Deadlock Avoidance

Requires that the system has some additional ***a priori*** information available

- Simplest and most useful model requires that each process declare the ***maximum number*** of resources of each type that it may need
- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition
- Resource-allocation *state* is defined by the number of available and allocated resources, and the maximum demands of the processes







# Safe State

- When a process requests an available resource, system must decide if immediate allocation leaves the system in a safe state
- System is in **safe state** if there exists a sequence  $\langle P_1, P_2, \dots, P_n \rangle$  of ALL the processes in the systems such that for each  $P_i$ , the resources that  $P_i$  can still request can be satisfied by currently available resources + resources held by all the  $P_j$ , with  $j < i$
- That is:
  - If  $P_i$  resource needs are not immediately available, then  $P_i$  can wait until all  $P_j$  have finished
  - When  $P_j$  is finished,  $P_i$  can obtain needed resources, execute, return allocated resources, and terminate
  - When  $P_i$  terminates,  $P_{i+1}$  can obtain its needed resources, and so on





# Basic Facts

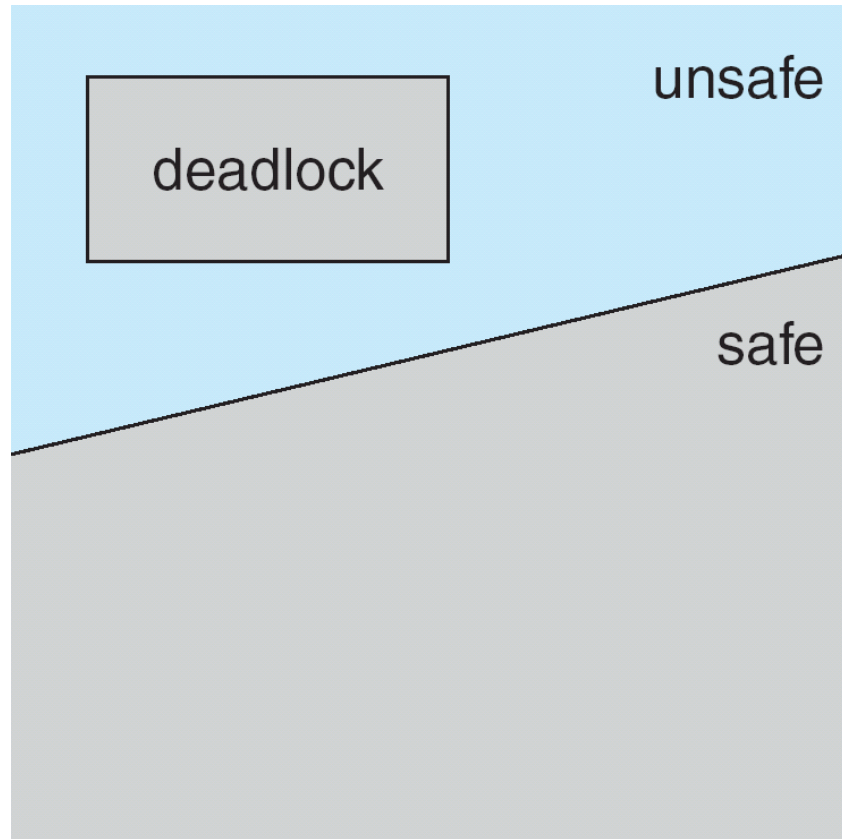
---

- If a system is in safe state  $\Rightarrow$  no deadlocks
- If a system is in unsafe state  $\Rightarrow$  possibility of deadlock
- Avoidance  $\Rightarrow$  ensure that a system will never enter an unsafe state.





# Safe, Unsafe, Deadlock State



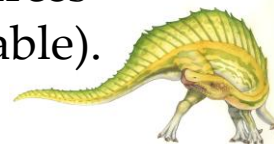


# Example

Consider a system with twelve resources and three threads:  $T_0$ ,  $T_1$ , and  $T_2$ . Thread  $T_0$  requires ten resources, thread  $T_1$  may need as many as four, and thread  $T_2$  may need up to nine resources. Suppose that, at time  $t_0$ , thread  $T_0$  is holding five resources, thread  $T_1$  is holding two resources, and thread  $T_2$  is holding two resources.

|       | <u>Maximum Needs</u> | <u>Current Needs</u> |
|-------|----------------------|----------------------|
| $T_0$ | 10                   | 5                    |
| $T_1$ | 4                    | 2                    |
| $T_2$ | 9                    | 2                    |

At time  $t_0$ , the system is in a safe state. The sequence  $\langle T_1, T_0, T_2 \rangle$  satisfies the safety condition. Thread  $T_1$  can immediately be allocated all its resources and then return them (the system will then have five available resources); then thread  $T_0$  can get all its resources and return them (the system will then have ten available resources); and finally thread  $T_2$  can get all its resources and return them (the system will then have all twelve resources available).





## Example Contd..

---

Suppose that, at time  $t_1$ , thread  $T_2$  requests and is allocated one more resource. The system is no longer in a safe state.

At this point, only thread  $T_1$  can be allocated all its resources.

When it returns them, the system will have only four available resources.

Since thread  $T_0$  is allocated five resources but has a maximum of ten, it may request five more resources.

If it does so, it will have to wait, because they are unavailable.

Similarly, thread  $T_2$  may request six additional resources and have to wait, resulting in a deadlock.

Our mistake was in granting the request from thread  $T_2$  for one more resource. If we had made  $T_2$  wait until either of the other threads had finished and released its resources, then we could have avoided the deadlock





# Avoidance Algorithms

---

- Single instance of a resource type
  - Use a resource-allocation graph
  
- Multiple instances of a resource type
  - Use the banker's algorithm





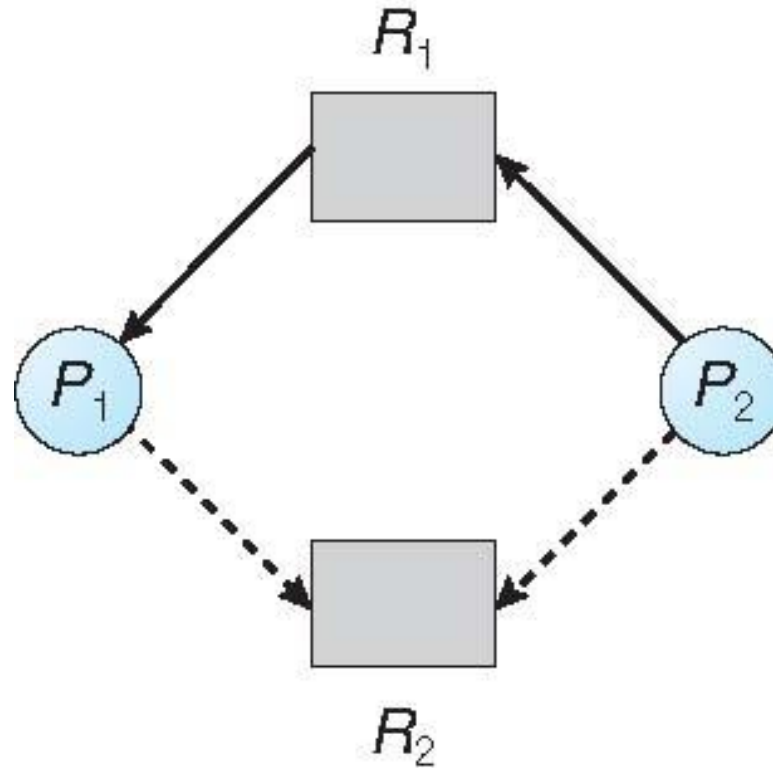
# Resource-Allocation Graph Scheme

- **Claim edge**  $P_i \rightarrow R_j$  indicated that process  $P_j$  may request resource  $R_j$ ; represented by a dashed line
- Claim edge converts to request edge when a process requests a resource
- Request edge converted to an assignment edge when the resource is allocated to the process
- When a resource is released by a process, assignment edge reconverts to a claim edge
- Resources must be claimed *a priori* in the system

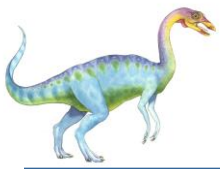




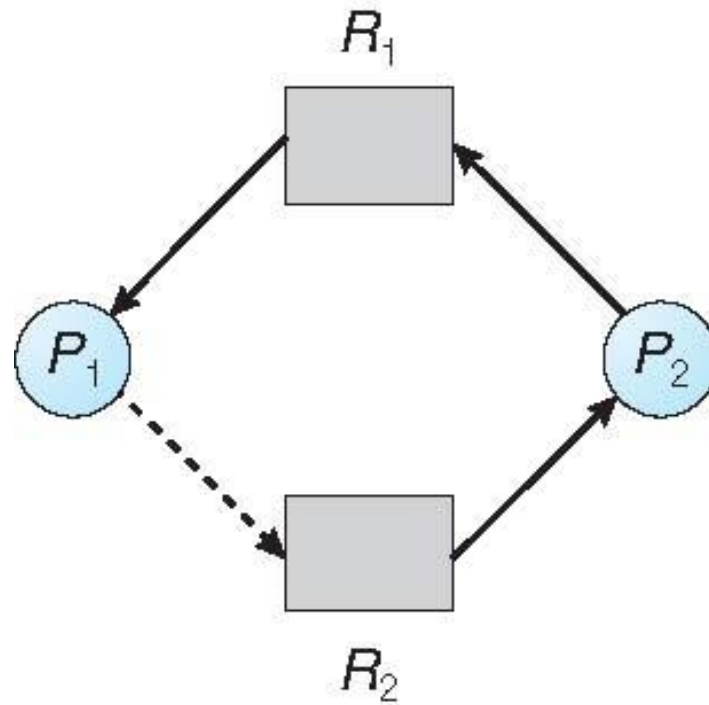
# Resource-Allocation Graph







# Unsafe State In Resource-Allocation Graph





# Resource-Allocation Graph Algorithm

---

- Suppose that process  $P_i$  requests a resource  $R_j$
- The request can be granted only if converting the request edge to an assignment edge does not result in the formation of a cycle in the resource allocation graph





# Banker's Algorithm

---

- ❑ Multiple instances
- ❑ Each process must a priori claim maximum use
- ❑ When a process requests a resource it may have to wait
- ❑ When a process gets all its resources it must return them in a finite amount of time





# Data Structures for the Banker's Algorithm

Let  $n$  = number of processes, and  $m$  = number of resources types.

- **Available:** Vector of length  $m$ . If available  $[j] = k$ , there are  $k$  instances of resource type  $R_j$  available
- **Max:**  $n \times m$  matrix. If  $Max[i, j] = k$ , then process  $P_i$  may request at most  $k$  instances of resource type  $R_j$
- **Allocation:**  $n \times m$  matrix. If  $Allocation[i, j] = k$  then  $P_i$  is currently allocated  $k$  instances of  $R_j$
- **Need:**  $n \times m$  matrix. If  $Need[i, j] = k$ , then  $P_i$  may need  $k$  more instances of  $R_j$  to complete its task

$$Need[i, j] = Max[i, j] - Allocation[i, j]$$





# Safety Algorithm

1. Let **Work** and **Finish** be vectors of length  $m$  and  $n$ , respectively.  
Initialize:

**Work = Available**

**Finish [i] = false for  $i = 0, 1, \dots, n-1$**

2. Find an  $i$  such that both:

(a) **Finish [i] = false**

(b) **Need<sub>i</sub> ≤ Work**

If no such  $i$  exists, go to step 4

3. **Work = Work + Allocation<sub>i</sub>**  
**Finish[i] = true**  
go to step 2

4. If **Finish [i] == true** for all  $i$ , then the system is in a safe state





# Resource-Request Algorithm for Process $P_i$

**$Request_i$**  = request vector for process  $P_i$ . If  **$Request_i[j] = k$**  then process  $P_i$  wants  $k$  instances of resource type  $R_j$

1. If  **$Request_i \leq Need_i$** , go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If  **$Request_i \leq Available$** , go to step 3. Otherwise  $P_i$  must wait, since resources are not available
3. Pretend to allocate requested resources to  $P_i$  by modifying the state as follows:

$$Available = Available - Request_i;$$

$$Allocation_i = Allocation_i + Request_i;$$

$$Need_i = Need_i - Request_i;$$

- If safe  $\Rightarrow$  the resources are allocated to  $P_i$
- If unsafe  $\Rightarrow P_i$  must wait, and the old resource-allocation state is restored





# Example of Banker's Algorithm

- 5 processes  $P_0$  through  $P_4$ ;

3 resource types:

$A$  (10 instances),  $B$  (5 instances), and  $C$  (7 instances)

- Snapshot at time  $T_0$ :

|       | <u>Allocation</u> | <u>Max</u>  | <u>Available</u> |
|-------|-------------------|-------------|------------------|
|       | $A \ B \ C$       | $A \ B \ C$ | $A \ B \ C$      |
| $P_0$ | 0 1 0             | 7 5 3       | 3 3 2            |
| $P_1$ | 2 0 0             | 3 2 2       |                  |
| $P_2$ | 3 0 2             | 9 0 2       |                  |
| $P_3$ | 2 1 1             | 2 2 2       |                  |
| $P_4$ | 0 0 2             | 4 3 3       |                  |





## Example (Cont.)

- The content of the matrix **Need** is defined to be **Max – Allocation**

|       | <u>Need</u> |   |   |
|-------|-------------|---|---|
|       | A           | B | C |
| $P_0$ | 7           | 4 | 3 |
| $P_1$ | 1           | 2 | 2 |
| $P_2$ | 6           | 0 | 0 |
| $P_3$ | 0           | 1 | 1 |
| $P_4$ | 4           | 3 | 1 |







## Find a Safe Sequence

Start with Available = (3, 3, 2)

### Check each process:

- **P0** needs (7,4,3) → Not  $\leq$  Available
- **P1** needs (1,2,2) → ☒ Can proceed
  - New Available = (3+2, 3+0, 2+0) = (5, 3, 2)
- **P2** needs (6,0,0) → Not  $\leq$  Available
- **P3** needs (0,1,1) → ☒ Can proceed
  - New Available = (5+2, 3+1, 2+1) = (7, 4, 3)
- **P0** needs (7,4,3) → ☒ Can proceed
  - New Available = (7+0, 4+1, 3+0) = (7, 5, 3)
- **P2** needs (6,0,0) → ☒ Can proceed
  - New Available = (7+3, 5+0, 3+2) = (10, 5, 5)
- **P4** needs (4,3,1) → ☒ Can proceed
  - New Available = (10+0, 5+0, 5+2) = (10, 5, 7)

☒ **Safe Sequence Found:**

**P1 → P3 → P0 → P2 → P4**





- **Let's try a different order:**
- **Initial Available: (3, 3, 2)**
- **P3** needs (0,1,1) → ✓ Can proceed
  - New Available = (5, 4, 3)
- **P1** needs (1,2,2) → ✓ Can proceed
  - New Available = (7, 4, 3)
- **P0** needs (7,4,3) → ✓ Can proceed
  - New Available = (7, 5, 3)
- **P2** needs (6,0,0) → ✓ Can proceed
  - New Available = (10, 5, 5)
- **P4** needs (4,3,1) → ✓ Can proceed
  - Final Available = (10, 5, 7)
- ✓ This gives us another valid safe sequence:
- **P3 → P1 → P0 → P2 → P4**





## Example: $P_1$ Request (1,0,2)

- Check that Request  $\leq$  Available (that is,  $(1,0,2) \leq (3,3,2) \Rightarrow$  true

|       | <u>Allocation</u> | <u>Need</u> | <u>Available</u> |
|-------|-------------------|-------------|------------------|
|       | A B C             | A B C       | A B C            |
| $P_0$ | 0 1 0             | 7 4 3       | 2 3 0            |
| $P_1$ | 3 0 2             | 0 2 0       |                  |
| $P_2$ | 3 0 2             | 6 0 0       |                  |
| $P_3$ | 2 1 1             | 0 1 1       |                  |
| $P_4$ | 0 0 2             | 4 3 1       |                  |

- Executing safety algorithm shows that sequence  $\langle P_1, P_3, P_4, P_0, P_2 \rangle$  satisfies safety requirement
- Can request for (3,3,0) by  $P_4$  be granted?
- Can request for (0,2,0) by  $P_0$  be granted?





# Exercise

Consider the following snapshot of a system:

**Allocation Max Available**

| $AB$  | $C$ | $D$ |   | $AB$ | $C$ | $D$ |   | $AB$ | $C$ | $D$ |   |   |
|-------|-----|-----|---|------|-----|-----|---|------|-----|-----|---|---|
| $T_0$ | 0   | 0   | 1 | 2    | 0   | 0   | 1 | 2    | 1   | 5   | 2 | 0 |
| $T_1$ | 1   | 0   | 0 | 0    | 1   | 7   | 5 | 0    |     |     |   |   |
| $T_2$ | 1   | 3   | 5 | 4    | 2   | 3   | 5 | 6    |     |     |   |   |
| $T_3$ | 0   | 6   | 3 | 2    | 0   | 6   | 5 | 2    |     |     |   |   |
| $T_4$ | 0   | 0   | 1 | 4    | 0   | 6   | 5 | 6    |     |     |   |   |

Answer the following questions using the banker's algorithm:

- What is the content of the matrix **Need**?
- Is the system in a safe state?
- If a request from thread *T1* arrives for (0,4,2,0), can the request be granted immediately?





# Exercise

Consider the following snapshot of a system:

| <b>Allocation</b> | <b>Max</b>    |
|-------------------|---------------|
| <i>AB C D</i>     | <i>AB C D</i> |
| <i>T0</i> 3 0 1 4 | 5 1 1 7       |
| <i>T1</i> 2 2 1 0 | 3 2 1 1       |
| <i>T2</i> 3 1 2 1 | 3 3 2 1       |
| <i>T3</i> 0 5 1 0 | 4 6 1 2       |
| <i>T4</i> 4 2 1 2 | 6 3 2 5       |

Using the banker's algorithm, determine whether or not each of the following states is unsafe. If the state is safe, illustrate the order in which the threads may complete. Otherwise, illustrate why the state is unsafe.

- a. **Available** = (0, 3, 0, 1)
- b. **Available** = (1, 0, 0, 2)



# End of Chapter 7

---

